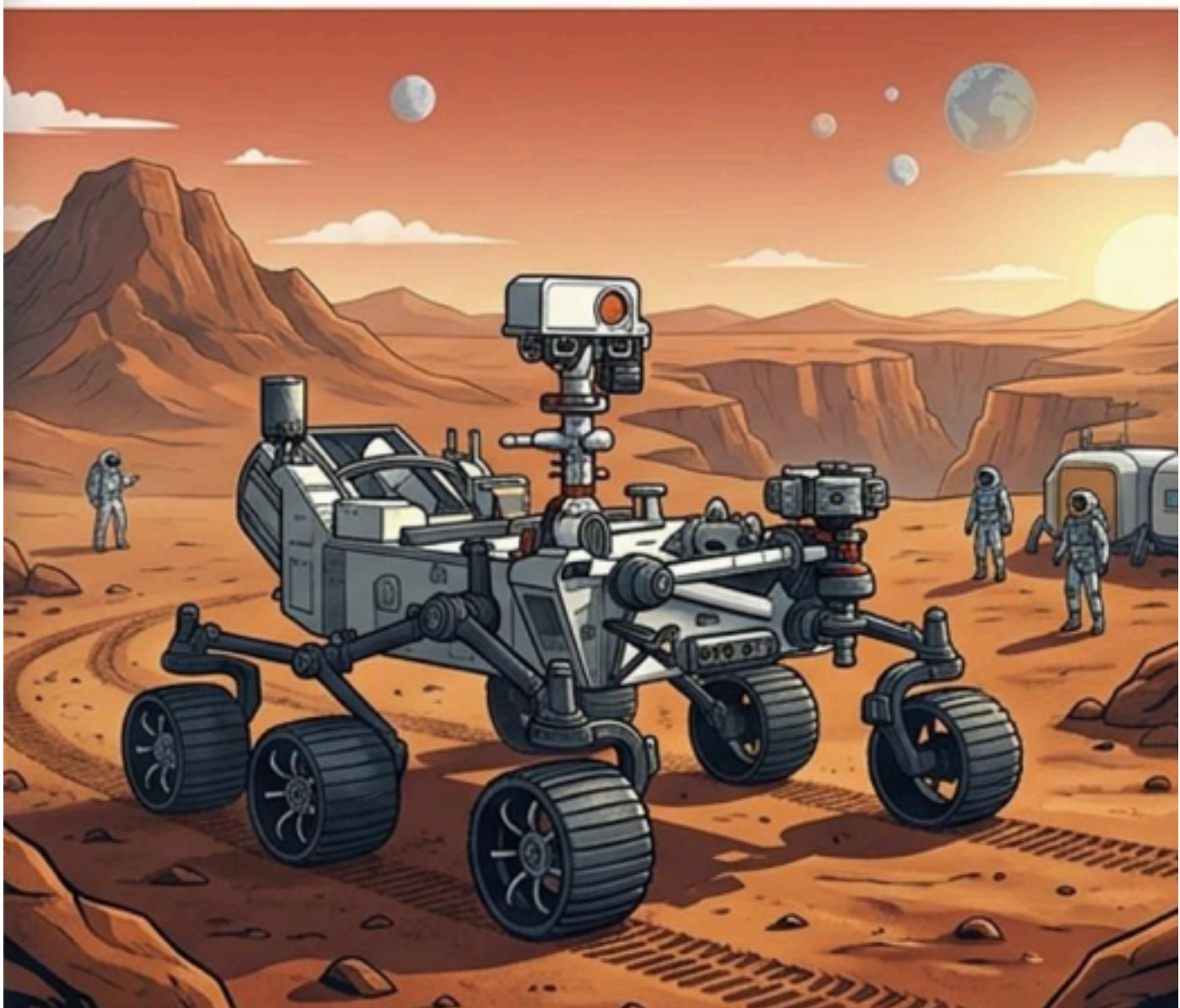


PROJETO: MISSÃO ROVER MARTE

DOCUMENTO DE REFERÊNCIA



Projeto: Rover de Marte - Sobrevivência e Exploração

O Objetivo

O programa deve simular um Rover que começa em uma posição (0,0) com uma quantidade limitada de **energia** e **integridade do casco**. O objetivo é chegar à base em (10,10) realizando cálculos de movimento e enfrentando eventos aleatórios.

Estrutura Sugerida para a Dupla

- **Integrante A (O Engenheiro de Sistemas):** Foca na lógica de movimentação, consumo de energia e entrada de dados do usuário.
- **Integrante B (O Especialista de Missão):** Foca nos cálculos de distância, eventos aleatórios (tempestades, crateras) e condições de vitória/derrota.

1. Visão Geral

Este projeto é um desafio de programação em Java desenvolvido para ser realizado em **duplas**. O objetivo é criar um simulador de console onde os alunos gerenciam um veículo de exploração espacial (Rover) em solo marciano.

O projeto foca na aplicação prática de:

- Estruturas de repetição (while).
- Estruturas de decisão (if/else, switch).
- Operadores aritméticos e lógicos.
- Uso da classe Math para cálculos geométricos.

2. O Desafio

O Rover inicia na posição coordenadas $(0, 0)$ e deve chegar à Base de Comando localizada em $(10, 10)$. No entanto, o ambiente é hostil:

1. **Energia:** Cada movimento consome energia. Se chegar a 0, a missão falha.
2. **Integridade:** Eventos aleatórios (tempestades e rochas) podem danificar o chassi.
3. **Navegação:** O usuário deve decidir a direção baseada no cálculo da distância real até o alvo.

3. Requisitos Técnicos

Para que o projeto seja considerado concluído, o código deve conter:

- **Loop de Jogo:** O programa não deve fechar até que o Rover vença ou seja destruído.

- **Cálculo Matemático:** Implementação da Distância Euclidiana:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **Entrada de Dados:** Leitura de comandos via teclado (W, A, S, D).
- **Fator Aleatório:** Uso da classe Random para simular perigos imprevistos.
- **Pesquisa sobre** Math.pow e Math.sqrt no Java para montar a lógica da fórmula acima

4. Sugestão de Divisão para a Dupla

- **Aluno A (Engenheiro de Sistemas):** Responsável por criar a estrutura do loop principal, o menu de comandos e a atualização das coordenadas de posição.
- **Aluno B (Especialista de Missão):** Responsável pela lógica de eventos aleatórios (tempestades), cálculo de distância e verificação das condições de vitória/derrota.

Um Desafio Extra

Para tornar o projeto realmente desafiador, adicionem uma **Mecânica de Terreno**:

- Crie uma variável para representar o "tipo de solo".
- Se o solo for "Arenoso", o movimento gasta o dobro de energia.
- Se for "Rochoso", há risco de perder integridade do casco.

Como dividir o trabalho?

Vocês podem criar uma classe **Rover** para guardar os dados e o **Main** para o loop do jogo. Enquanto um escreve a lógica de "onde o robô está", o outro escreve a lógica de "o que acontece com o robô".

Aqui está um roteiro "sem mistérios" para vocês montarem a lógica do desenvolvimento:

Como montar seu jogo: O Guia Simplificado

Imagine que você está criando um jogo de tabuleiro digital. O Rover é a

sua peça e o console é o seu tabuleiro. Para o código funcionar, pensem nele em **4 partes**:

1. A Mochila do Rover (Variáveis)

Antes de começar, vocês precisam anotar os itens de sobrevivência. No código, isso são as variáveis:

- **Onde ele está?** x e y (começam em 0).
- **Quanto ele aguenta?** energia e vida (começam em 100).
- **Onde ele quer chegar?** O alvo é o ponto (10, 10).

2. O Motor do Jogo (O Loop while)

O jogo é um ciclo que se repete. A lógica é: **"Enquanto o robô não quebrar e não ficar sem pilha, o jogo continua"**.

- Dentro desse ciclo, o programa faz três coisas: Mostra o status, pede um comando e calcula o que aconteceu.

3. O Volante (Movimentação)

O aluno "A" pode cuidar disso. É como um controle de videogame:

- Se o jogador apertar **W**, o robô vai para frente (sobe no mapa).
- Se apertar **D**, ele vai para a direita.
- **Importante:** Cada vez que ele anda, ele gasta um pouco de energia. Não dá para andar para sempre!

4. Os Imprevistos (Sorte ou Azar)

O aluno "B" cuida do "destino". O computador vai "jogar um dado" (usando o Random).

- Se o dado cair em um número baixo, uma tempestade de areia atingiu o robô e ele perde vida.
- Isso serve para que o jogador tenha pressa de chegar ao objetivo antes que o robô estrague.

Dica Visual para os alunos:

O mapa funciona como um plano cartesiano da escola:

- **Eixo Y (W e S):** Para frente e para trás.

- **Eixo X (A e D):** Para os lados.
- **Distância:** É como se eles estivessem usando um GPS que diz quantos metros faltam para chegar na lanchonete (a base).

O "Pulo do Gato":

Imprimam no console (`System.out.println`) cada passo que o robô der. Se os números estiverem mudando, não saberão se o código está funcionando!

Exemplo de conversa na dupla:

"Ei, eu faço a parte de ele andar para os lados, você faz a parte de calcular se a gente bateu em uma pedra?"